

Java Multithreading & Concurrency

Interview Revision Notes: synchronized, locks, race condition, deadlock, thread pool APIs

How to use this PDF

Read the WHY first, then the HOW. Keep the diagrams in mind, and use the interview questions to test yourself quickly.

Roadmap

- Race condition - why shared data goes wrong
- synchronized - how Java protects shared code
- Object lock vs Class lock
- Lock interface - ReentrantLock basics
- Deadlock, livelock, starvation
- Thread pool APIs - ExecutorService, Callable, Future, submit, shutdown

1. Race Condition

Why it happens

When two or more threads read and update the same shared data at the same time, the final value can become wrong. This is called a race condition.

Shared balance = 1000

Thread A reads 1000

Thread B reads 1000

Thread A adds 500 -> 1500

Thread B subtracts 300 -> 700

Expected final value: 1200

But actual final value may be: 700 or 1500 or something else

Real-life example

Two people writing the same notebook at the same time can overwrite each other's changes. The last write may win, and important data can be lost.

Interview line

Race condition is a timing problem. The result depends on the order in which threads execute.

2. synchronized keyword

Why we need it

synchronized lets only one thread enter the critical section at a time. It protects shared data from race conditions.

```
class Counter {  
    private int count = 0;  
  
    public synchronized void increment() {  
        count++;  
    }  
}
```

How it works

Thread A enters synchronized method

Thread B waits

Thread A exits

Thread B enters

Simple idea

Only one thread gets the key to the room. Other threads wait outside until the key is returned.

Important note

synchronized can be used on a method or on a block. A synchronized block is usually better when you want to lock only a small part of the code.

3. Object lock vs Class lock

Object lock = lock on one particular object
Class lock = lock on the Class object itself

Object lock

Used for instance methods or synchronized blocks on this. Two different objects can run at the same time.

Class lock

Used for static synchronized methods. The lock belongs to the class, so all objects share the same lock.

```
class A {  
    public synchronized void m1() {} // object lock  
    public static synchronized void m2() {} // class lock  
}
```

Interview line

Object lock protects one object. Class lock protects the class level shared state.

4. Lock interface and ReentrantLock

Why use Lock?

Java provides Lock when you need more control than synchronized. The most common implementation is ReentrantLock.

```
Lock lock = new ReentrantLock();  
lock.lock();  
try {  
    // critical section  
} finally {  
    lock.unlock();  
}
```

- Must unlock in finally block.
- Can try to lock without waiting forever using tryLock().
- Can make fairness decisions in some cases.

Real-life idea

It is like having a security guard with more options: wait, try later, or walk away if the room is busy.

5. Deadlock, livelock, starvation

Deadlock

Two threads wait for each other forever. Thread A has Lock 1 and wants Lock 2. Thread B has Lock 2 and wants Lock 1. Nobody moves.

```
Thread A -> Lock 1 held -> wants Lock 2  
Thread B -> Lock 2 held -> wants Lock 1  
Result: both wait forever
```

Real-life example

Two people stand in a narrow hallway and both refuse to move first. The hallway stays blocked.

Livelock

Threads are not blocked, but they keep changing state and still do no useful work. They are active, but progress is zero.

Starvation

One thread keeps waiting because other threads keep getting priority. It is unfair scheduling, not a deadlock.

Deadlock = waiting forever
Livelock = moving, but no progress
Starvation = unfairly waiting too long

Interview line

Deadlock is the one most commonly asked. The classic prevention idea is to avoid circular waiting, acquire locks in the same order, and keep lock scope small.

6. Thread pool APIs

Why thread pools?

Creating a new thread for every task is expensive. A thread pool reuses already created threads, which improves performance and control.

```
ExecutorService service = Executors.newFixedThreadPool(4);
service.submit(task1);
service.submit(task2);
service.shutdown();
```

Main APIs

- `Executors.newFixedThreadPool(n)` - fixed number of threads
- `Executors.newCachedThreadPool()` - grows when needed
- `Executors.newSingleThreadExecutor()` - one worker thread
- `submit()` - returns `Future`
- `execute()` - fire and forget
- `shutdown()` - stop accepting new tasks
- `shutdownNow()` - try to stop immediately

Callable vs Runnable

`Runnable` -> no return value, no checked exception
`Callable` -> returns value, can throw exception

```
Future<String> future = service.submit(callableTask);
String result = future.get(); // waits for the answer
```

Real-life example

A restaurant keeps multiple waiters ready. New customers do not force the restaurant to hire a new waiter every time.

7. Mini interview revision

Question	My Answer	Correct Answer
What is race condition?	Shared data is updated by multiple threads at the same time	Wrong result caused by timing
What does <code>synchronized</code> do?	Allows only one thread to enter the protected section	Prevents race condition
What is deadlock?	Threads wait forever for each other	No one can continue
Why use a thread pool?	Reuse threads instead of creating new ones again and again	Better performance and control

8. What to learn next

- `volatile` - visibility between threads
- `AtomicInteger` and atomic classes
- `ConcurrentHashMap` and other concurrent collections
- `CompletableFuture` for async programming
- `CountDownLatch` and `Semaphore` basics
- `ReadWriteLock` if you want deeper lock control

Final memory chain

Shared data
↓
Race condition
↓
synchronized / Lock
↓
Deadlock prevention
↓
Thread pool
↓
Better performance and cleaner code

Quick Quiz Answer Key

Question	Correct Answer	Why
Race condition	Unsafely shared mutable data	Timing changes the output
synchronized	One thread at a time	Creates mutual exclusion
Object lock	Lock on this object	Only that instance is protected
Class lock	Lock on Class object	Shared by all instances
Deadlock	Circular waiting forever	Each thread waits for the other
Thread pool	Reusable worker threads	Improves performance